

DATA ANALYTICS LAB-R20

Laboratory Manual

III YEAR B.TECH-I SEM(CSD)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (DATA
SCIENCE)

 **MLR**
Institute of Technology
Dundigal, Hyderabad-500043

LAB MANUAL
DATA ANALYTICS LAB-
A6DS02



Department of CSE (Data Science)

WEEK-1

Working with R

- Reading different types of data sets (.txt, .csv) from web and desk and writing in file in specified disk location
- Reading Excel data sheet in R
- Reading XML dataset in R

1. Reading Different Types of Datasets Text files (.txt)

```
data_txt <- read.table("file.txt", header = TRUE)
```

CSV files (.csv)

```
data_csv <- read.csv("file.csv", header = TRUE)
```

From Web

```
data_web <- read.csv("https://example.com/data.csv")
```

Writing to Disk

```
write.csv(data_csv, "output.csv", row.names = FALSE)
```

2. Reading Excel Data Sheet in R

```
library(readxl)
```

```
data_excel <- read_excel("file.xlsx", sheet = 1)
```

3. Reading XML Dataset in R

```
library(XML)
```

```
data_xml <- xmlParse("file.xml")
```

```
root_node <- xmlRoot(data_xml)
```

WEEK-2

WEEK-2 LEARNING NUMPY

- Creating arrays
- Using arrays and scalars
- Indexing Arrays
- Transposition
- Array Processing
- Array Input and Output

1. Creating Arrays

- Arrays are created using `np.array()`.
- Other functions:
 - `np.zeros(n)` → array of zeros.
 - `np.ones(n)` → array of ones.
 - `np.arange(start, stop, step)` → range of values.
 - `np.linspace(start, stop, num)` → evenly spaced values.

```
arr = np.array([1, 2, 3, 4])
```

```
print(arr)
```

2. Using Arrays and Scalars

- NumPy supports vectorized operations.
- Operations are applied element-wise.
- Scalars can be added, multiplied, or divided with arrays directly.

```
arr = np.array([1, 2, 3])
```

```
print(arr + 5) # Adds 5 to each element
```

3. Indexing Arrays

- Access elements using indices.
- Supports slicing (`arr[start:end]`).
- Can use boolean indexing and fancy indexing.

```
arr = np.array([10, 20, 30, 40])
```

```
print(arr[2]) # Single element
```

```
print(arr[1:3]) # Slice
```

4. Transposition

- Used to flip rows and columns in 2D arrays.
- `arr.T` → transpose of the array.

```
matrix = np.array([[1,2,3],[4,5,6]])
```

```
print(matrix.T)
```

5. Array Processing

- Common operations:
 - `sum()`, `mean()`, `std()`, `min()`, `max()`.
 - Element-wise operations: `+`, `-`, `*`, `/`.
- Works across rows (`axis=0`) or columns (`axis=1`).

Example:

```
arr = np.array([[1,2,3],[4,5,6]])
```

```
print(arr.sum(axis=0)) # Column-wise sum
```

6. Array Input and Output

- Save and load arrays:
 - `np.save('file.npy', arr)` → save array.
 - `np.load('file.npy')` → load array.
 - Text format:
 - `np.savetxt('file.txt', arr)` → save as text.
 - `np.loadtxt('file.txt')` → load from text.
-

WEEK-3: INTRODUCTION TO PANDAS PART-I

- Series
- DataFrames
- Index objects
- Reindex
- Drop Entry
- Selecting Entries

1. Series

- A Series is a one-dimensional labeled array.
- Can hold any data type (integers, strings, floats, etc.).
- Labels are called the index.
- Created using `pd.Series(data, index=...)`.

Example:

```
import pandas as pd  
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])  
print(s)
```

2. DataFrames

- A DataFrame is a two-dimensional labeled data structure (like a table).
- Consists of rows and columns.
- Each column is a Series.
- Created using dictionaries, lists, NumPy arrays, or external files (CSV, Excel).

Example:

```
data = {'Name': ['Tom', 'Jerry'], 'Age': [25, 22]}  
df = pd.DataFrame(data)  
print(df)
```

3. Index Objects

- The Index is used to label rows and columns.
- Provides identity to data and supports alignment.
- Immutable (cannot be changed directly).
- Accessed using `df.index` or `df.columns`.

4. Reindex

- Used to change the order or add new labels to an existing Series/DataFrame.
- Missing labels are filled with NaN.

Example:

```
s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
```

```
s2 = s.reindex(['a', 'b', 'c', 'd'])
```

```
print(s2)
```

5. Drop Entry

- Removes rows or columns from a Series/DataFrame.
- drop() method is used.
- Can specify axis=0 (rows) or axis=1 (columns).

Example:

```
python
```

```
df = pd.DataFrame({'A':[1,2,3], 'B':[4,5,6]})
```

```
df2 = df.drop('A', axis=1)
```

```
print(df2)
```

6. Selecting Entries

- Access data using:
 - Label-based indexing: loc[]
 - Position-based indexing: iloc[]
- Can select single values, rows, or columns.

```
df = pd.DataFrame({'Name':['Tom','Jerry'], 'Age':[25,22]})
```

```
print(df.loc[0, 'Name']) # Label-based
```

```
print(df.iloc[1, 1])    # Position-based
```

WEEK-4: INTRODUCTION TO PANDAS PART-II

- Data Alignment
- Rank and Sort
- Summary Statistics
- Missing Data
- Index Hierarchy

1. Data Alignment

- Pandas aligns data automatically based on index labels.
- Operations between Series/DataFrames match values by index.
- Missing labels are filled with NaN.

```
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
```

```
s2 = pd.Series([4, 5, 6], index=['b', 'c', 'd'])
```

```
print(s1 + s2)
```

2. Rank and Sort

- Sorting
 - `sort_index()` → sort by index.
 - `sort_values()` → sort by values.
- Ranking
 - `rank()` assigns ranks to values.
 - Handles ties with methods: average, min, max, first.

3. Summary Statistics

- Common functions: `mean()`, `median()`, `std()`, `min()`, `max()`, `sum()`.
- `describe()` → quick summary (count, mean, std, min, quartiles, max).
- Works across rows (`axis=0`) or columns (`axis=1`).

4. Missing Data

- Missing values are represented as NaN.
- Detection: `isnull()`, `notnull()`.
- Handling:
 - `dropna()` → remove missing values.

- `fillna(value)` → replace missing values.
- Methods: `ffill` (forward fill), `bfill` (backward fill).

5. Index Hierarchy (MultiIndex)

- Pandas supports hierarchical indexing (MultiIndex).
- Allows multiple levels of indexing for rows/columns.

Example:

python

```
arrays = [['A', 'A', 'B', 'B'], [1, 2, 1, 2]]
```

```
index = pd.MultiIndex.from_arrays(arrays, names=('letter', 'number'))
```

```
df = pd.DataFrame(np.random.randn(4, 2), index=index)
```

- Enables complex data selection and reshaping (`stack()`, `unstack()`).

WEEK-5 | WORKING WITH DATA PART-I

- Reading and Writing Text Files
- JSON with Python
- HTML with Python
- Microsoft Excel files with Python

I. Reading and writing a Text Files

with open("example.txt", "w") as file:

```
# Write content to the file
```

```
file.write("Hello, this is a sample text.\n")
```

```
file.write("Writing to a file in Python is easy!\n")
```

```
file.write("You can write multiple lines as well.\n")
```

```
print("File 'example.txt' has been written.")
```

```
# Open a file in read mode ("r")
```

with open("example.txt", "r") as file:

```
# Read the entire content of the file
```

```
content = file.read()
```

```
# Alternatively, you can read line by line
```

```
# lines = file.readlines()
```

```
# Print the content of the file
```

```
print("Content of the file:")
```

```
print(content)
```

I. JSON with Python

```
import json
```

```
# Sample data
```

```
data = {  
    "name": "John",  
    "age": 30,  
    "city": "New York"  
}
```

```
# Open a file in write mode ("w") for writing JSON
with open("example.json", "w") as json_file:
    # Write the JSON data to the file
    json.dump(data, json_file)

print("JSON data has been written to 'example.json'.")

import json

# Open a file in read mode ("r") for reading JSON
with open("example.json", "r") as json_file:
    # Load the JSON data from the file
    loaded_data = json.load(json_file)

# Print the loaded JSON data
print("Loaded JSON data:")
print(loaded_data)
```

HTML with PythonMicrosoft

```
pip install beautifulsoup4 requests
```

```
import requests from bs4
```

```
import BeautifulSoup
```

```
# URL of the website you want to fetch HTML from
```

```
url = "https://www.example.com"
```

```
# Send a GET request to the URL
```

```
response = requests.get(url)
```

```
# Check if the request was successful (status code 200)
```

```
if response.status_code == 200:
```

```
    # Parse the HTML content using BeautifulSoup
```

```
    soup = BeautifulSoup(response.text, 'html.parser')
```

```
# Print the HTML content or perform further processing
```

```
print(soup.prettify())

else:

    print(f'Failed to fetch HTML. Status code: {response.status_code}')

# HTML content to write to the file

html_content = """

<!DOCTYPE html>

<html>

<head>

    <title>My HTML Page</title>

</head>

<body>

    <h1>Hello, HTML!</h1>

    <p>This is a simple HTML page written with Python.</p>

</body>

</html>

"""

# Write HTML content to a file

with open("output.html", "w") as html_file:

    html_file.write(html_content)

print("HTML content has been written to 'output.html'.")
```

Excel file with Python

```
import pandas as pd

# Read an Excel file

df = pd.read_excel("example.xlsx")

# Display the DataFrame (contents of the Excel file)

print("DataFrame from Excel:")

print(df)

# Create a sample DataFrame
```

```
data = {  
    'Name': ['John', 'Alice', 'Bob'],  
    'Age': [25, 30, 22],  
    'City': ['New York', 'San Francisco', 'Seattle']  
}
```

Writing

```
df_to_write = pd.DataFrame(data)  
# Write DataFrame to an Excel file  
df_to_write.to_excel("output.xlsx", index=False)  
print("DataFrame has been written to 'output.xlsx'.")
```

WEEK-6 | WORKING WITH DATA PART-II

- Merge
- Combining DataFrames
- Pivoting
- Duplicates in DataFrames
- Mapping
- Binning
- Outliers

```
import pandas as pd
```

```
# Create two sample DataFrames
```

```
df1 = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Name': ['Alice', 'Bob', 'Charlie']  
})
```

```
df2 = pd.DataFrame({  
    'ID': [2, 3, 4],  
    'Age': [25, 30, 22]  
})
```

```
# Merge DataFrames based on the 'ID' column
```

```
merged_df = pd.merge(df1, df2, on='ID', how='inner')
```

```
print("Merged DataFrame:")
```

```
print(merged_df)
```

```
# Merge based on multiple columns
```

```
merged_df = pd.merge(df1, df2, on=['ID'], how='inner')
```

```
Combining DataFrame
```

```
import pandas as pd
```

```
# Create two sample DataFrames
```

```
df1 = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Name': ['Alice', 'Bob', 'Charlie']  
})
```

```
df2 = pd.DataFrame({  
    'ID': [4, 5, 6],  
    'Name': ['David', 'Eva', 'Frank']  
})
```

```
# Concatenate DataFrames along rows
```

```
concatenated_df = pd.concat([df1, df2], ignore_index=True)
```

```
print("Concatenated DataFrame along rows:")
```

```
print(concatenated_df)
```

```
# Create two sample DataFrames
```

```
df1 = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Name': ['Alice', 'Bob', 'Charlie']  
})
```

```
df2 = pd.DataFrame({  
    'Age': [25, 30, 22],  
    'City': ['New York', 'San Francisco', 'Seattle']  
})
```

Concatenate DataFrames along columns

```
concatenated_df = pd.concat([df1, df2], axis=1)
```

```
print("Concatenated DataFrame along columns:")
```

```
print(concatenated_df)
```

Pivoting

import pandas as pd

Create a sample DataFrame

```
data = {  
    'Date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02'],  
    'Category': ['A', 'B', 'A', 'B'],  
    'Sales': [100, 150, 200, 250]  
}
```

```
df = pd.DataFrame(data)
```

Display the original DataFrame

```
print("Original DataFrame:")
```

```
print(df)
```

Pivot the DataFrame

```
pivot_df = df.pivot(index='Date', columns='Category', values='Sales')
```

Display the pivoted DataFrame

```
print("\nPivoted DataFrame:")
```

```
print(pivot_df)
```

Duplicates

```
import pandas as pd

# Create a sample DataFrame with duplicate values in the 'Name' column
data = {
    'ID': [1, 2, 3, 4, 5],
    'Name': ['Alice', 'Bob', 'Charlie', 'Bob', 'Alice'],
    'Age': [25, 30, 22, 30, 25]
}

df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Identify duplicate rows based on the 'Name' column
duplicates = df.duplicated(subset='Name')

# Display the rows that are duplicates based on the 'Name' column
print("\nDuplicate Rows based on 'Name':")
print(df[duplicates])

# Drop duplicate rows based on the 'Name' column
df_no_duplicates = df.drop_duplicates(subset='Name')

# Display the DataFrame without duplicates based on the 'Name' column
print("\nDataFrame without Duplicates based on 'Name':")
print(df_no_duplicates)

Mapping
import pandas as pd
```

```
# Create a sample DataFrame
```

```
data = {  
    'Numbers': [1, 2, 3, 4, 5]  
}
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")  
print(df)
```

```
# Define a function to square a number
```

```
def square(x):  
    return x ** 2
```

```
# Use the apply function to apply the square function to the 'Numbers' column
```

```
df['Squared'] = df['Numbers'].apply(square)
```

```
# Display the DataFrame with squared numbers
```

```
print("\nDataFrame with Squared Numbers:")  
print(df)
```

```
df['Squared'] = df['Numbers'].apply(lambda x: x ** 2)
```

```
binning
```

```
import pandas as pd
```

```
# Create a sample DataFrame with numerical data
```

```
data = {  
    'Scores': [75, 82, 95, 68, 60, 90, 78, 88, 72, 85]  
}
```

```
df = pd.DataFrame(data)
```

```
# Define bin edges and labels
```

```
bins = [0, 60, 70, 80, 90, 100]
```

```
labels = ['F', 'D', 'C', 'B', 'A']
```

```
# Create a new column 'Grade' by binning the 'Scores' column
```

```
df['Grade'] = pd.cut(df['Scores'], bins=bins, labels=labels, right=False)
```

```
# Display the original DataFrame and the DataFrame with the 'Grade' column
```

```
print("Original DataFrame:")
```

```
print(df)
```

Outlier

```
import pandas as pd
```

```
from scipy.stats import zscore
```

```
# Create a sample DataFrame with numerical data
```

```
data = {  
    'Values': [10, 15, 12, 18, 22, 8, 25, 30, 5, 35, 40]  
}
```

```
df = pd.DataFrame(data)
```

```
# Calculate Z-scores for each value in the 'Values' column
```

```
z_scores = zscore(df['Values'])
```

```
# Define a threshold for identifying outliers (e.g., Z-score greater than 3 or less than -3)
```

```
threshold = 3
```

```
# Identify outliers based on the threshold
```

```
outliers = (z_scores > threshold) | (z_scores < -threshold)
```

```
# Add a new column 'Is_Outlier' to the DataFrame indicating whether each value is an outlier
```

```
df['Is_Outlier'] = outliers
```

```
# Display the original DataFrame and the DataFrame with the 'Is_Outlier' column
```

```
print("Original DataFrame:")
```

```
print(df)
```

WEEK-7 | WORKING WITH DATA PART-III

- GroupBy on DataFrames
- Splitting Applying and Combining
- Cross Tabulation

Group by on Data Frame

```
import pandas as pd
```

```
# Create a sample DataFrame
```

```
data = {  
    'Category': ['A', 'B', 'A', 'B', 'A', 'B'],  
    'Value': [10, 15, 20, 25, 30, 35]  
}
```

```
df = pd.DataFrame(data)
```

```
# Group by 'Category' and calculate the mean for each group
```

```
grouped_df = df.groupby('Category').mean()
```

```
# Display the original DataFrame and the result of the groupby operation
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
print("\nGrouped DataFrame (mean for each category):")
```

```
print(grouped_df)
```

Splitting Applying and Combining

Splitting: The `groupby('Category')` operation is used to split the DataFrame into groups based on the 'Category' column.

Applying: The ['Sales'].sum() operation is applied to each group to calculate the sum of sales for each category.

Combining: The result is a Pandas Series that represents the total sales for each category.

```
import pandas as pd

# Create a sample DataFrame
data = {
    'Category': ['A', 'B', 'A', 'B', 'A', 'B'],
    'Sales': [100, 150, 200, 120, 180, 220]
}

df = pd.DataFrame(data)

# Step 1: Splitting - Group by 'Category'
grouped_df = df.groupby('Category')

# Step 2: Applying - Calculate the sum for each group
sum_per_category = grouped_df['Sales'].sum()

# Step 3: Combining - Display the result
print("Total Sales per Category:")
print(sum_per_category)

# Calculate multiple aggregations at once
result = grouped_df['Sales'].agg(['sum', 'mean', 'count'])

# Display the result
print("Aggregations per Category:")
print(result)
```

Cross Tabulation

pd.crosstab() function to perform cross-tabulation on a DataFrame. Cross-tabulation is a way to summarize and analyze the relationship between two or more categorical variables.

```
import pandas as pd
```

```
# Create a sample DataFrame
```

```
data = {  
    'Category': ['A', 'B', 'A', 'B', 'A', 'B'],  
    'Color': ['Red', 'Blue', 'Red', 'Green', 'Blue', 'Red']  
}
```

```
df = pd.DataFrame(data)
```

```
# Perform cross-tabulation
```

```
cross_tab = pd.crosstab(df['Category'], df['Color'])
```

```
# Display the cross-tabulation result
```

```
print("Cross-Tabulation:")
```

```
print(cross_tab)
```

```
# Specify additional parameters for the crosstab function
```

```
cross_tab = pd.crosstab(df['Category'], df['Color'], margins=True, margins_name='Total')
```

```
# Display the modified cross-tabulation result
```

```
print("Modified Cross-Tabulation:")
```

```
print(cross_tab)
```

WEEK-8 | DATA VISUALIZATION

- Installing Seaborn
- Histograms
- Combining Plot Styles

pip install seaborn

import seaborn as sns

Histograms

pip install seaborn matplotlib

import seaborn as sns

import matplotlib.pyplot as plt

Sample data

data = [5, 10, 15, 20, 25, 30, 30, 35, 40, 45, 50]

Create a histogram using seaborn

sns.histplot(data, bins=10, kde=False, color='skyblue')

Add labels and title

plt.xlabel('Values')

plt.ylabel('Frequency')

plt.title('Histogram Example')

Show the plot

plt.show()

Combining Plot Styles

import seaborn as sns

import matplotlib.pyplot as plt

Set the style to 'darkgrid'

sns.set(style='darkgrid')

Sample data

data = sns.load_dataset('tips')

Create a figure with two subplots

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))

```
# Plot a histogram on the first subplot
```

```
sns.histplot(data['total_bill'], bins=20, kde=True, color='skyblue', ax=axes[0])
```

```
axes[0].set_title('Histogram')
```

```
# Plot a scatter plot on the second subplot
```

```
sns.scatterplot(x='total_bill', y='tip', data=data, color='coral', ax=axes[1])
```

```
axes[1].set_title('Scatter Plot')
```

```
# Adjust layout
```

```
plt.tight_layout()
```

```
# Show the combined plot
```

```
plt.show()
```

WEEK-9 | DATA VISUALIZATION

- Box and Violin Plots
- Regression Plots
- Heatmaps and Clustered Matrices
- TabPy: Tableau with Python

Box and Violin Plots

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
data = sns.load_dataset('tips')
```

```
# Set the style to 'whitegrid'
```

```
sns.set(style='whitegrid')
```

```
# Create a box plot
```

```
plt.figure(figsize=(8, 6))
```

```
sns.boxplot(x='day', y='total_bill', data=data, palette='Set2')
```

```
plt.title('Box Plot of Total Bill by Day')
```

```
plt.show()
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
data = sns.load_dataset('tips')
```

```
# Set the style to 'whitegrid'
```

```
sns.set(style='whitegrid')
```

```
# Create a violin plot
plt.figure(figsize=(8, 6))
sns.violinplot(x='day', y='total_bill', data=data, palette='Pastel1')
plt.title('Violin Plot of Total Bill by Day')
plt.show()
```

regression plots

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Sample data
data = sns.load_dataset('tips')
```

```
# Set the style to 'whitegrid'
sns.set(style='whitegrid')
```

```
# Create a box plot
plt.figure(figsize=(8, 6))
sns.boxplot(x='day', y='total_bill', data=data, palette='Set2')
plt.title('Box Plot of Total Bill by Day')
plt.show()
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Sample data
data = sns.load_dataset('tips')
```

```
# Set the style to 'whitegrid'
```

```
sns.set(style='whitegrid')
```

```
# Create a violin plot
```

```
plt.figure(figsize=(8, 6))
```

```
sns.violinplot(x='day', y='total_bill', data=data, palette='Pastell')
```

```
plt.title('Violin Plot of Total Bill by Day')
```

```
plt.show()
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
data = sns.load_dataset('tips')
```

```
# Set the style to 'whitegrid'
```

```
sns.set(style='whitegrid')
```

```
# Create a scatter plot with a regression line using lmlot
```

```
sns.lmplot(x='total_bill', y='tip', data=data, height=6, aspect=1.5, scatter_kws={'s': 30},  
line_kws={'color': 'blue'})
```

```
plt.title('Scatter Plot with Regression Line')
```

```
plt.show()
```

```
heatmaps and clustered matrice
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
data = sns.load_dataset('flights')
```

```
# Pivot the data to create a matrix
```

```
flights_matrix = data.pivot_table(index='month', columns='year', values='passengers')
```

```
# Set the style to 'whitegrid'
```

```
sns.set(style='whitegrid')
```

```
# Create a heatmap
```

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(flights_matrix, cmap='YlGnBu', annot=True, fmt='d', linewidths=.5)
```

```
plt.title('Flights Data - Heatmap')
```

```
plt.show()
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
data = sns.load_dataset('flights')
```

```
# Pivot the data to create a matrix
```

```
flights_matrix = data.pivot_table(index='month', columns='year', values='passengers')
```

```
'whitegrid'
```

```
sns.set(style='whitegrid')
```

```
# Create a clustered matrix
```

```
plt.figure(figsize=(10, 8))
```

```
sns.clustermap(flights_matrix, cmap='YlGnBu', annot=True, fmt='d', linewidths=.5)
```

```
plt.title('Flights Data - Clustered Matrix')
```

```
plt.show()
```

WEEK-10 | USE CASES

- Data Project – Stock Market Analysis
- Data Project – Customer Segmentation
- Data Project – Credit Card Fraud Detection

Stock market analysis ,customer segmentation , credit card fraud
detection

pip install yfinance pandas matplotlib

import yfinance as yf

import pandas as pd

import matplotlib.pyplot as plt

def stock_analysis(ticker, start_date, end_date):

 # Download historical stock data

 stock_data = yf.download(ticker, start=start_date, end=end_date)

 # Calculate daily returns

 stock_data['Daily_Return'] = stock_data['Adj Close'].pct_change()

 # Calculate cumulative returns

 stock_data['Cumulative_Return'] = (1 + stock_data['Daily_Return']).cumprod()

 # Plotting

 plt.figure(figsize=(10, 6))

Plot stock prices

 plt.subplot(2, 1, 1)

 plt.plot(stock_data['Adj Close'])

 plt.title(f'{ticker} Stock Price')

 plt.xlabel('Date')

 plt.ylabel('Stock Price')

Plot cumulative returns

 plt.subplot(2, 1, 2)

```
plt.plot(stock_data['Cumulative_Return'], color='r')
```

```
plt.title(f'{ticker} Cumulative Returns')
```

```
plt.xlabel('Date')
```

```
plt.ylabel('Cumulative Returns')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
if __name__ == "__main__":
```

```
    # Input stock symbol (ticker), start date, and end date
```

```
    stock_ticker = input("Enter stock symbol (e.g., AAPL): ")
```

```
    start_date = input("Enter start date (YYYY-MM-DD): ")
```

```
    end_date = input("Enter end date (YYYY-MM-DD): ")
```

```
    # Perform stock analysis
```

```
    stock_analysis(stock_ticker, start_date, end_date)
```

```
pip install pandas numpy matplotlib scikit-learn
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Generate some random customer data for demonstration purposes
```

```
np.random.seed(42)
```

```
data = {
```

```
    'Age': np.random.randint(18, 65, 100),
```

```
    'Income': np.random.randint(20000, 100000, 100),
```

```
}
```

```
df = pd.DataFrame(data)

# Standardize the data

scaler = StandardScaler()

scaled_data = scaler.fit_transform(df)

# Determine the optimal number of clusters using the Elbow Method

wcss = []

for i in range(1, 11):

    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10,
random_state=0)

    kmeans.fit(scaled_data)

    wcss.append(kmeans.inertia_)

# Plot the Elbow Method graph

plt.plot(range(1, 11), wcss)

plt.title('Elbow Method for Optimal k')

plt.xlabel('Number of clusters')

plt.ylabel('WCSS') # Within-Cluster Sum of Squares

plt.show()

# Based on the Elbow Method, choose the optimal number of clusters (k)

k_optimal = int(input("Enter the optimal number of clusters (k): "))

# Apply k-means clustering

kmeans = KMeans(n_clusters=k_optimal, init='k-means++', max_iter=300, n_init=10,
random_state=0)

df['Cluster'] = kmeans.fit_predict(scaled_data)

# Display the segmented customers

print("\nSegmented Customers:")

print(df.groupby('Cluster').mean())
```

Visualize the clusters

```
plt.scatter(scaled_data[:, 0], scaled_data[:, 1], c=df['Cluster'], cmap='viridis')
```

```
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=300, c='red')
```

```
plt.title('Customer Segmentation')
```

```
plt.xlabel('Scaled Age')
```

```
plt.ylabel('Scaled Income')
```

```
plt.show()
```

```
pip install pandas scikit-learn
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import LocalOutlierFactor
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
# Load the credit card fraud dataset (replace with your dataset)
```

```
df = pd.read_csv('credit_card_fraud_dataset.csv')
```

```
# Separate features and labels
```

```
X = df.drop('Class', axis=1)
```

```
y = df['Class']
```

```
# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Standardize the features
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# Train the Local Outlier Factor model
```

```
model = LocalOutlierFactor(contamination=0.01)
```

```
y_pred = model.fit_predict(X_test)
```

```
# Convert the predictions (-1 for anomalies, 1 for normal) to binary labels (0 for normal, 1 for fraud)
```

```
y_pred_binary = [1 if pred == -1 else 0 for pred in y_pred]
```

```
# Evaluate the model
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_binary))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_binary))
```